SUNSHINE WALLET

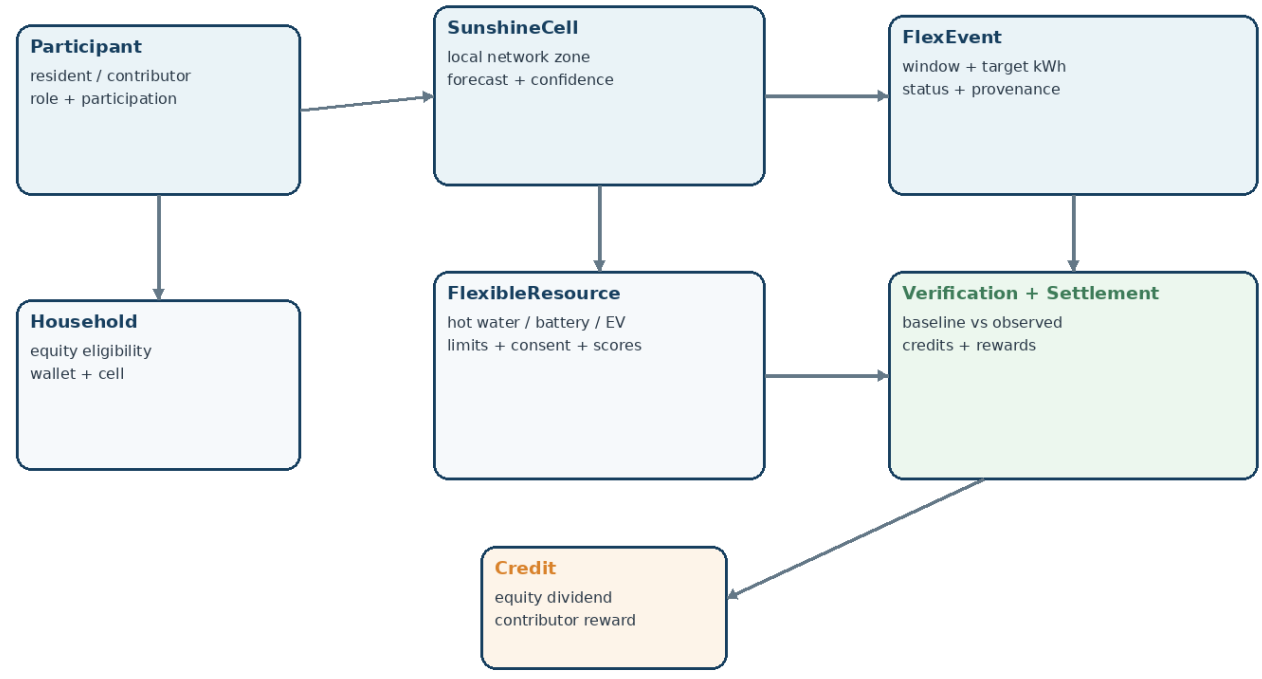
DOMAIN & DATA MODEL CONTRACT

Resident + Contributor + Council / Operator + Engine

Contract goal

Define one shared source of truth for the objects used by the Sunshine Wallet 25-hour hackathon MVP: identities, Sunshine Cells, flexible resources, solar contributors, events, dispatch decisions, simulation outputs, M&V, contributor attribution, settlement, credits, provenance and audit metadata.

Sunshine Wallet MVP - Domain Model Map



Version 1.0 | 22 August 2026

1. What This Model Contract Is

This is the shared domain contract that all three developers code against. It is intentionally narrower than a production electricity-market data model. The goal is to make the demo internally consistent, prevent frontend/backend type drift, and preserve the distinctions we have repeatedly refined: kW vs kWh, flexible demand vs solar contribution, planned vs actual response, baseline vs observed demand, verification vs settlement, and equity credits vs contributor rewards.

One-source-of-truth rule

Do not create separate frontend and backend versions of Participant, Resource, FlexEvent or Settlement. Put shared model definitions in one place and import them everywhere.

In scope

- Domain entities and value objects required by the MVP.
- Allowed enums and state transitions.
- Required fields, optional fields, units and provenance.
- Validation and invariants that must never be violated.
- Derived/calculated fields and which layer owns them.
- Example TypeScript interfaces and JSON records.
- Recommended persistence mapping for seeded JSON now and PostgreSQL later.
- Model ownership, contract tests and change-control rules.

Out of scope

- Production NEM settlement schemas.
- Full DNSP topology or phase modelling.
- Actual retailer billing schemas.
- Identity/KYC, OAuth or production RBAC.
- Regulatory-grade M&V methodology.
- Full tariff and payment processing.

2. Modelling Principles

Principle	Meaning	Contract consequence
Energy vs power	Event quantities are kWh; instantaneous/device power limits are kW.	targetFlexEnergyKwh, maxShiftEnergyKwh, maxPowerKw
Solar ≠ dispatchable load	Rooftop solar is measured as contributor export; the optimiser primarily dispatches flexible demand.	SolarContributor separate from FlexibleResource
No electron tracing	We attribute value by accounting within a qualifying event/cell, not by claiming Maria's electrons reached Dinh.	ContributorAttribution
Plan ≠ outcome	Scheduled energy and simulated/observed response are separate fields.	scheduledEnergyKwh vs actualResponseEnergyKwh
Baseline is counterfactual	Baseline is what would likely have happened without the event.	BaselineEstimate
Verify before settle	No settlement until M&V passes confidence gate.	Verification.settlementAllowed
Money in cents	Avoid floating-point money errors.	amountCents, valueRateCentsPerKwh
Truthfulness	Every external/synthetic field carries provenance.	DataProvenance
Explainability	Decisions store reasons and score components.	DispatchDecision.scoreBreakdown + rejectReason

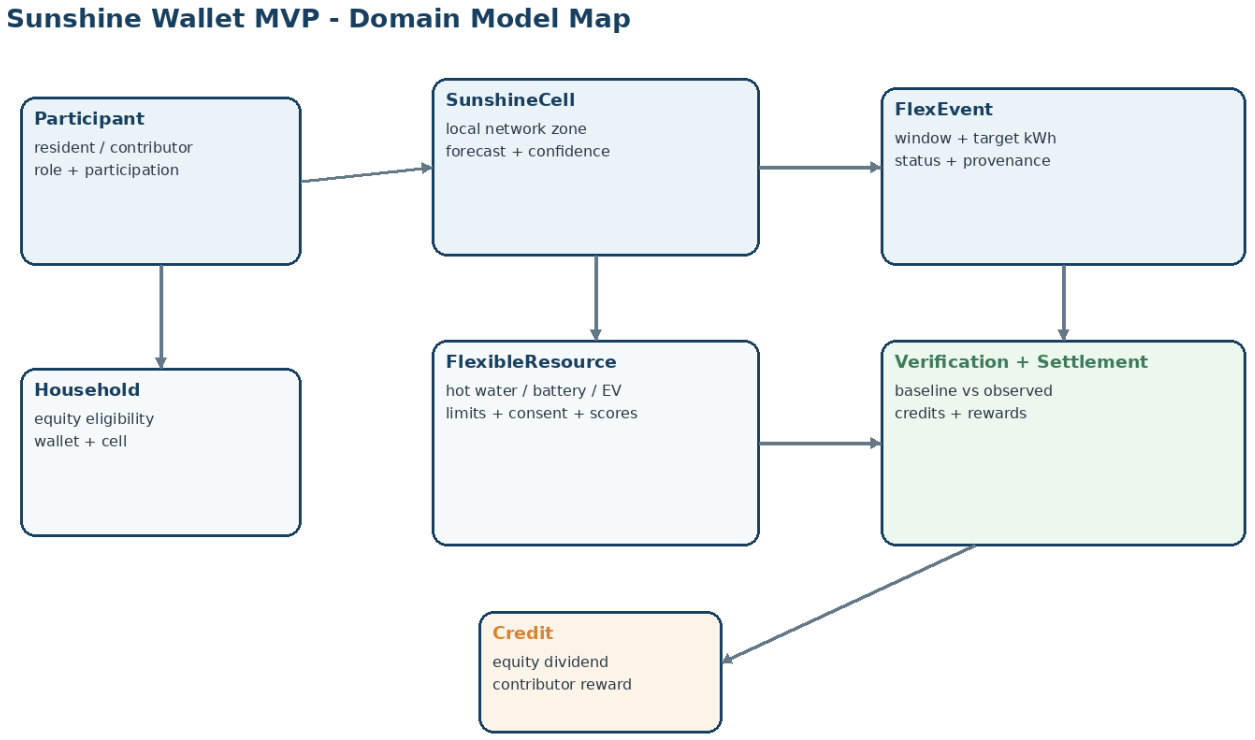
3. Canonical Enums

```
export type ParticipantRole =
  | 'EQUITY_PARTICIPANT'
  | 'CONTRIBUTOR'
  | 'BOTH';

export type ParticipationStatus = 'ACTIVE' | 'PAUSED' | 'OPTED_OUT';

export type ResourceType =
  | 'STORAGE_HOT_WATER'
  | 'COMMUNITY_BATTERY'
  | 'EV_CHARGER'
  | 'APARTMENT_COMMON_LOAD';

export type CompatibilityStatus = 'PENDING' | 'COMPATIBLE' | 'INCOMPATIBLE';

export type EventStatus =
  | 'DRAFT' | 'PROPOSED' | 'OPTIMISED' | 'DISPATCHED'
  | 'VERIFIED' | 'SETTLEMENT_BLOCKED' | 'SETTLED'
  | 'CREDITS_APPLIED' | 'CANCELLED';

export type DispatchDecisionType = 'SELECTED' | 'PARTIALLY_SELECTED' | 'REJECTED';

export type RejectReason =
  | 'NO_CONSENT' | 'WRONG_CELL' | 'UNAVAILABLE'
  | 'INCOMPATIBLE' | 'COMFORT_CONSTRAINT'
  | 'LOW_CONFIDENCE' | 'NOT_NEEDED';

export type DataProvenance =
  | 'REAL_PUBLIC' | 'PARTNER_VERIFIED' | 'ESTIMATED'
  | 'SIMULATED' | 'MOCKED' | 'CALCULATED';

export type CreditType = 'EQUITY_DIVIDEND' | 'CONTRIBUTOR_REWARD';
```

Important

The MVP should not invent new enum strings inside UI components. If a new state/reason is needed, change this contract first, then update all consumers.

4. Domain Map and Cardinality



Relationship	Cardinality	Rule
Household -> Participant	1 -> 1..n	A household may expose one demo participant, but model allows multiple people.
SunshineCell -> Household	1 -> many	A household belongs to one MVP Sunshine Cell at a time.
Household -> FlexibleResource	1 -> 0..many	A household may have no controllable asset and still be equity eligible.
Participant -> SolarContributorProfile	1 -> 0..1	Only contributor/BOTH roles need a contributor profile.
SunshineCell -> FlexEvent	1 -> many	Each event is scoped to exactly one cell.
FlexEvent -> DispatchDecision	1 -> many	One decision per evaluated resource for auditability.
FlexEvent -> Verification	1 -> 0..1	Created after dispatch/simulation.
FlexEvent -> Settlement	1 -> 0..1	Only if M&V gate allows.
Settlement -> Credit	1 -> many	Equity and contributor credits are ledger entries.

5. Household and Participant

Household

Household is the physical/program account context. It owns eligibility, cell membership and optional flexible resources. Keep sensitive detail out of the MVP; store a simple eligibility result/token rather than simulated welfare records.

```
export interface Household {
  id: string;
  displayLabel: string;
  suburb: string;
  sunshineCellId: string;
  householdType: 'RENTER' | 'OWNER' | 'APARTMENT' | 'OTHER';
  equityEligible: boolean;
  eligibilityToken?: string;
  eligibilityProvenance: DataProvenance;
  walletId: string;
}
```

Participant

```
export interface Participant {
  id: string;
  householdId: string;
  displayName: string;
  role: ParticipantRole;
  participationStatus: ParticipationStatus;
  globalConsentActive: boolean;
  createdAt: string;
  updatedAt: string;
}
```

Invariants

- EQUITY_PARTICIPANT requires Household.equityEligible = true.
- CONTRIBUTOR or BOTH may exist with no flexible load if the person contributes rooftop solar export.
- PAUSED/OPTED_OUT participants cannot be selected for a new dispatch.
- Resident UI may display names; operator views should prefer household/resource IDs where possible.

6. SunshineCell

A Sunshine Cell is the local electrical coordination zone used to decide whether a resource and solar contribution are relevant to the same event. In the hackathon, DAPTO-01 is synthetic/approximate. The model must never imply that suburb boundaries equal real electrical topology.

```
export interface SunshineCell {
  id: string;                    // e.g. cell_dapto_01
  displayName: string;           // DAPTO-01
  suburb: string;                // Dapto
  topologyRef: string;           // synthetic reference in MVP
  topologyProvenance: DataProvenance;

  equityScore: number;           // 0..1
  solarOpportunityScore: number; // 0..1
  networkNeedScore: number;      // 0..1
  overallConfidence: number;     // 0..1

  active: boolean;
}
```

Key invariant

Wrong cell is a hard rejection

A resource with sunshineCellId != event.sunshineCellId is rejected before scoring, no matter how high its equity or controllability score is.

7. FlexibleResource

FlexibleResource represents something the optimiser may schedule: storage hot water, battery charging, EV charging or an apartment common load. Rooftop solar export is deliberately not represented as this type.

```
export interface FlexibleResource {
  id: string;
  householdId?: string;          // optional for shared/community assets
  sunshineCellId: string;
  type: ResourceType;

  maxPowerKw: number;            // instantaneous power limit
  maxShiftEnergyKwh: number;     // event-window energy limit
  available: boolean;
  compatibilityStatus: CompatibilityStatus;
  consentActive: boolean;
  comfortSafe: boolean;

  responseFactor: number;        // 0..1 deterministic MVP simulation

  scores: {
    networkEffectiveness: number; // 0..1
    equityNeed: number;            // 0..1
    controllability: number;       // 0..1
    rotationFairness: number;      // 0..1
  };

  provenance: {
    compatibility: DataProvenance;
    availability: DataProvenance;
    limits: DataProvenance;
  };
}
```

Validation

- maxPowerKw > 0.
- maxShiftEnergyKwh >= 0.
- responseFactor in [0,1].
- All score components in [0,1].
- INCOMPATIBLE resources cannot be selected.
- A storage-hot-water record should expose comfortSafe; the optimiser may not bypass it.

8. SolarContributorProfile and Export Measurement

Solar contributors create/participate in the solar opportunity; they are not “turned on” by the optimiser. The profile stores enrolment and links to event-period export measurements used for accounting attribution.

```
export interface SolarContributorProfile {
  participantId: string;
  householdId: string;
  sunshineCellId: string;
  participationStatus: ParticipationStatus;
  exportMeterRef?: string;
  exportMeasurementProvenance: DataProvenance;
}

export interface ContributorExportInterval {
  participantId: string;
  eventId: string;
  intervalStart: string;
  intervalEnd: string;
  generationEnergyKwh?: number;
  householdUseEnergyKwh?: number;
  netExportEnergyKwh: number;
  provenance: DataProvenance;
}
```

Rule

No electron tracing

netExportEnergyKwh proves an accounting contribution at the connection point/event window. It does not prove those exact electrons reached a specific receiver.

9. EnergyInterval and BaselineReference

The MVP needs interval records to support window selection, charts and M&V. Keep event energy (kWh) and instantaneous/average power (kW) explicit.

```
export interface EnergyInterval {
  sunshineCellId: string;
  intervalStart: string;
  intervalEnd: string;

  forecastSolarExportEnergyKwh: number;
  comfortableExportEnergyKwh: number;
  baselineDemandEnergyKwh: number;
  observedDemandEnergyKwh?: number;

  forecastConfidence: number;
  eventRan: boolean;
  provenance: DataProvenance;
}

export interface BaselineReferenceDay {
  id: string;
  date: string;
  sunshineCellId: string;
  windowStartLocal: string;
  windowEndLocal: string;
  eventPeriodEnergyKwh: number;
  eventRan: boolean;
  comparable: boolean;
  provenance: DataProvenance;
}
```

Baseline invariant

Exclude contaminated reference days

For the simple MVP baseline, only comparable reference days with eventRan = false are eligible. If too few clean days exist, lower confidence or block settlement.

10. FlexEvent

FlexEvent is the aggregate/root object for one Sunshine Event. It owns the selected time window, target flexibility, lifecycle state and references to downstream results.

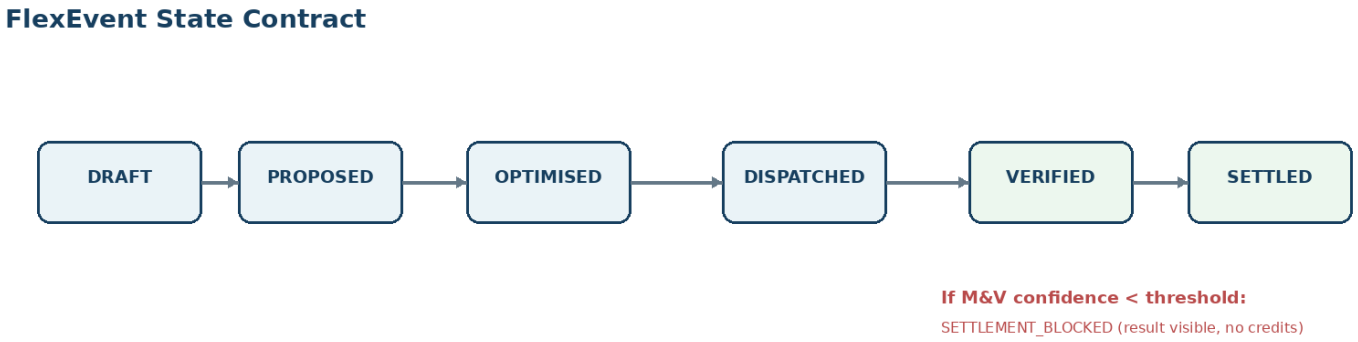
```
export interface FlexEvent {
  id: string;
  sunshineCellId: string;
  startTime: string;
  endTime: string;

  forecastExportEnergyKwh: number;
  comfortableExportEnergyKwh: number;
  targetFlexEnergyKwh: number;   // derived = max(0, forecast - comfortable)
  availableFlexEnergyKwh: number;
  forecastConfidence: number;

  status: EventStatus;
  simulationMode: boolean;
  createdAt: string;
  updatedAt: string;

  provenance: {
    networkCondition: DataProvenance;
    forecast: DataProvenance;
    target: 'CALCULATED';
  };
}
```

Derived target rule

```
targetFlexEnergyKwh = Math.max(
  0,
  forecastExportEnergyKwh - comfortableExportEnergyKwh
);
```

For the demo: 70 kWh forecast export - 46 kWh comfortable export = 24 kWh target flexible energy.

11. FlexEvent State Contract



State	Meaning
DRAFT	Inputs assembled; no event recommendation yet.
PROPOSED	Window passed forecast/network gates.
OPTIMISED	Dispatch decisions generated.
DISPATCHED	Simulated/partner dispatch executed.
VERIFIED	M&V complete and confidence gate passed.
SETTLEMENT_BLOCKED	M&V result visible but confidence below threshold.
SETTLED	Value pools and participant allocations calculated.
CREDITS_APPLIED	Ledger credits committed idempotently.
CANCELLED	Event cancelled/no longer actionable.

Illegal transition examples

DRAFT -> SETTLED is invalid. OPTIMISED -> CREDITS_APPLIED is invalid. Credits require verification and settlement first.

12. DispatchDecision and ScoreBreakdown

```
export interface ScoreBreakdown {
  networkEffectiveness: number;
  equityNeed: number;
  controllability: number;
  rotationFairness: number;
  weightedScore: number;
}

export interface DispatchDecision {
  eventId: string;
  resourceId: string;
  decision: DispatchDecisionType;
  rejectReason?: RejectReason;

  maxShiftEnergyKwh: number;
  scheduledEnergyKwh: number;
  scoreBreakdown?: ScoreBreakdown;

  explanation: string;
}
```

Score formula

```
weightedScore =
  0.35 * networkEffectiveness +
  0.30 * equityNeed +
  0.20 * controllability +
  0.15 * rotationFairness;
```

Selection invariant

A rejected resource has scheduledEnergyKwh = 0. A partially selected resource has 0 < scheduledEnergyKwh < maxShiftEnergyKwh. Selection stops when cumulative scheduled energy reaches the event target or eligible capacity is exhausted.

13. ResourceResponse and EventSimulation

The simulator represents the external physical systems we cannot control in the hackathon. It must be deterministic so repeated demos with the same inputs yield the same outputs.

```
export interface ResourceResponse {
  eventId: string;
  resourceId: string;
  scheduledEnergyKwh: number;
  responseFactor: number;
  actualResponseEnergyKwh: number; // scheduled * responseFactor
  status: 'RESPONDED' | 'FAILED';
  provenance: 'SIMULATED' | 'PARTNER_VERIFIED';
}

export interface EventSimulation {
  eventId: string;
  expectedFlexEnergyKwh: number;
  actualFlexResponseEnergyKwh: number;
  responses: ResourceResponse[];
  simulatedAt: string;
}
```

Demo example

Resource	Scheduled kWh	Factor	Actual kWh
HW-101	5.0	0.96	4.80
HW-102	4.0	0.975	3.90
BAT-01	10.0	0.97	9.70
APT-01	5.0	0.84	4.20
Total	24.0	-	22.60

14. BaselineEstimate and Verification (M&V)

M&V asks: compared with what would normally have happened without Sunshine Wallet, how much additional useful event-period demand occurred?

```
export interface BaselineEstimate {
  eventId: string;
  method: 'MEAN_COMPARABLE_NON_EVENT_DAYS';
  referenceDayIds: string[];
  baselineEnergyKwh: number;
  confidence: number;
  provenance: 'ESTIMATED' | 'SIMULATED';
}

export interface Verification {
  eventId: string;
  baselineEnergyKwh: number;
  observedEnergyKwh: number;
  verifiedFlexEnergyKwh: number;
  confidence: number;
  minimumSettlementConfidence: number;
  settlementAllowed: boolean;
  status: 'VERIFIED' | 'PROVISIONAL';
}
```

Calculation

```
verifiedFlexEnergyKwh = Math.max(
  0,
  observedEnergyKwh - baselineEnergyKwh
);

settlementAllowed =
  confidence >= minimumSettlementConfidence;
```

Example: baseline 50.0 kWh, observed 72.6 kWh -> verified flexibility 22.6 kWh. At 82% confidence with a 70% gate, settlement is allowed.

15. ContributorAttribution

Contributor attribution divides only the verified/qualifying outcome. It first computes each contributor's share of qualifying export, then caps total attributable energy at the verified accommodated amount.

```
export interface ContributorAttribution {
  eventId: string;
  participantId: string;
  qualifyingExportEnergyKwh: number;
  qualifyingExportShare: number;      // 0..1
  attributedEnergyKwh: number;
  contributorPoolShare: number;       // usually same as qualifyingExportShare in MVP
}

export interface ContributorAttributionSummary {
  eventId: string;
  totalQualifyingExportEnergyKwh: number;
  attributableEnergyCapKwh: number;
  attributions: ContributorAttribution[];
}
```

Calculation

```
totalExport = sum(contributor.qualifyingExportEnergyKwh);
cap = Math.min(totalExport, verifiedAccommodatedEnergyKwh);

share_i = export_i / totalExport;
attributedEnergy_i = share_i * cap;
```

Example: Maria exports 6.5 of 13.0 qualifying kWh = 50%. If verified accommodation is capped at 10 kWh, Maria is attributed 5 kWh for settlement accounting - not because her exact electrons were traced.

16. SettlementPolicy and Settlement

Settlement converts verified value into an Equity Pool, Contributor Pool and optional Community Reserve. The policy is explicit and versioned.

```
export interface SettlementPolicy {
  id: string;
  version: string;
  equityFloor: number;               // e.g. 0.60
  equityPoolShare: number;            // e.g. 0.65
  contributorPoolShare: number;       // e.g. 0.30
  reserveShare: number;               // e.g. 0.05
  valueRateCentsPerKwh: number;       // simulated MVP rate
  minimumMvConfidence: number;        // e.g. 0.70
}

export interface Settlement {
  id: string;
  eventId: string;
  policyId: string;
  verifiedProgramValueCents: number;
  equityPoolCents: number;
  contributorPoolCents: number;
  reserveCents: number;
  equityFloorSatisfied: boolean;
  status: 'VALID' | 'BLOCKED';
  createdAt: string;
}
```

Policy invariants

- equityPoolShare + contributorPoolShare + reserveShare = 1.0.
- equityPoolShare >= equityFloor.
- Verification.settlementAllowed must be true.
- Money is integer cents.
- valueRateCentsPerKwh is labelled SIMULATED in the MVP.

17. Credit and Wallet

A credit is the auditable ledger outcome shown in the resident/contributor wallet. For equity receivers, the bill/program credit is the mechanism that lowers effective electricity cost. For contributors, the reward is a separate program credit.

```
export interface Credit {
  id: string;
  eventId: string;
  settlementId: string;
  participantId: string;
  householdId: string;
  type: CreditType;
  amountCents: number;
  reason: string;
  status: 'PENDING' | 'APPLIED' | 'VOID';
  policyVersion: string;
  idempotencyKey: string;
  createdAt: string;
}

export interface WalletSummary {
  participantId: string;
  month: string; // YYYY-MM
  equityDividendCents: number;
  contributorRewardCents: number;
  totalBenefitCents: number;
  latestCredit?: Credit;
}
```

No double-counting rule

Receiver benefit

Do not model “cheaper midday energy” and an Equity Dividend as two automatic benefits for the same event. For the MVP, the program/bill credit is the mechanism that reduces the receiver's effective bill.

18. CompatibilityCheck and Partner-Mock Models

The prototype simulates checks with authorised parties. The model must clearly expose that the response is mocked, while allowing real engine logic to consume it.

```
export interface CompatibilityCheckRequest {
  householdId: string;
  requestedDeviceType: ResourceType;
  consentToCheck: boolean;
}

export interface CompatibilityCheckResult {
  checkId: string;
  householdId: string;
  deviceType: ResourceType;
  compatible: boolean;
  compatibilityStatus: CompatibilityStatus;
  maxPowerKw?: number;
  maxShiftEnergyKwh?: number;
  controlAvailable: boolean;
  providerLabel: 'SIMULATED_AUTHORISED_PARTNER';
  provenance: 'MOCKED';
  checkedAt: string;
}
```

UI language

Show “Prototype compatibility check - simulated authorised-partner response,” not “Connected to Endeavour Energy” or any claim of a live integration.

19. Provenance and Audit Metadata

Truthfulness is part of the product. Provenance belongs in the model so the UI does not guess.

```
export interface ProvenanceRecord {
  source: DataProvenance;
  sourceLabel?: string;
  generatedAt?: string;
  notes?: string;
}

export interface AuditEvent {
  id: string;
  entityType: string;
  entityId: string;
  action: string;
  actorType: 'SYSTEM' | 'OPERATOR' | 'PARTICIPANT';
  actorId?: string;
  timestamp: string;
  metadata?: Record<string, unknown>;
}
```

Field/data	MVP provenance
Sunshine Cell topology	SIMULATED
Historical meter intervals	SIMULATED
Device compatibility	MOCKED
Optimiser score/selection	CALCULATED
Event response	SIMULATED
Baseline	ESTIMATED from SIMULATED history
M&V result	CALCULATED
Settlement rate	SIMULATED
Credits	CALCULATED ledger

20. Complete TypeScript Contract - Aggregates

```
export interface EventAggregate {
  event: FlexEvent;
  cell: SunshineCell;
  evaluatedResources: FlexibleResource[];
  dispatchDecisions: DispatchDecision[];
  simulation?: EventSimulation;
  baseline?: BaselineEstimate;
  verification?: Verification;
  contributorAttribution?: ContributorAttributionSummary;
  settlement?: Settlement;
  credits: Credit[];
}

export interface DemoScenario {
  households: Household[];
  participants: Participant[];
  cells: SunshineCell[];
  resources: FlexibleResource[];
  contributors: SolarContributorProfile[];
  historicalReferenceDays: BaselineReferenceDay[];
  energyIntervals: EnergyInterval[];
  events: FlexEvent[];
  policies: SettlementPolicy[];
}
```

Recommended boundary

The engine should accept domain objects and return domain results. UI components should not recalculate optimiser scores, M&V or settlement themselves.

21. Canonical JSON - DAPTO-01 Demo Scenario


```
{
  "event": {
    "id": "evt_001",
    "sunshineCellId": "cell_dapto_01",
    "startTime": "2026-08-23T12:00:00+10:00",
    "endTime": "2026-08-23T14:00:00+10:00",
    "forecastExportEnergyKwh": 70,
    "comfortableExportEnergyKwh": 46,
    "targetFlexEnergyKwh": 24,
    "availableFlexEnergyKwh": 31,
    "forecastConfidence": 0.82,
    "status": "PROPOSED",
    "simulationMode": true,
    "provenance": {
      "networkCondition": "SIMULATED",
      "forecast": "SIMULATED",
      "target": "CALCULATED"
    }
  },
  "policy": {
    "id": "policy_equity_01",
    "version": "1.0",
    "equityFloor": 0.60,
    "equityPoolShare": 0.65,
    "contributorPoolShare": 0.30,
    "reserveShare": 0.05,
    "valueRateCentsPerKwh": 80,
    "minimumMvConfidence": 0.70
  }
}
```

22. Canonical JSON - Resource + Dispatch Result


```
{
  "resource": {
    "id": "HW-101",
    "householdId": "hh_dinh",
    "sunshineCellId": "cell_dapto_01",
    "type": "STORAGE_HOT_WATER",
    "maxPowerKw": 3.6,
    "maxShiftEnergyKwh": 5,
    "available": true,
    "compatibilityStatus": "COMPATIBLE",
    "consentActive": true,
    "comfortSafe": true,
    "responseFactor": 0.96,
    "scores": {
      "networkEffectiveness": 0.95,
      "equityNeed": 0.90,
      "controllability": 1.00,
      "rotationFairness": 0.80
    }
  },
  "decision": {
    "eventId": "evt_001",
    "resourceId": "HW-101",
    "decision": "SELECTED",
    "maxShiftEnergyKwh": 5,
    "scheduledEnergyKwh": 5,
    "scoreBreakdown": {
      "networkEffectiveness": 0.95,
      "equityNeed": 0.90,
      "controllability": 1.00,
      "rotationFairness": 0.80,
      "weightedScore": 0.9225
    },
    "explanation": "Eligible, high network value and equity need."
  }
}
```

23. Canonical JSON - M&V, Attribution and Credits


```
{
  "verification": {
    "eventId": "evt_001",
    "baselineEnergyKwh": 50.0,
    "observedEnergyKwh": 72.6,
    "verifiedFlexEnergyKwh": 22.6,
    "confidence": 0.82,
    "minimumSettlementConfidence": 0.70,
    "settlementAllowed": true,
    "status": "VERIFIED"
  },
  "mariaAttribution": {
    "eventId": "evt_001",
    "participantId": "p_maria",
    "qualifyingExportEnergyKwh": 6.5,
    "qualifyingExportShare": 0.50,
    "attributedEnergyKwh": 5.0,
    "contributorPoolShare": 0.50
  },
  "credits": [
    {
      "participantId": "p_dinh",
      "type": "EQUITY_DIVIDEND",
      "amountCents": 411
    },
    {
      "participantId": "p_maria",
      "type": "CONTRIBUTOR_REWARD",
      "amountCents": 271
    }
  ]
}
```

24. Validation and Cross-Entity Invariants

Invariant	Requirement
Event target	targetFlexEnergyKwh >= 0 and equals the calculated network-opportunity target for MVP.
Cell consistency	Resource, contributor and event must share the relevant Sunshine Cell to qualify.
Consent	No dispatch when participant/resource consent is inactive.
Comfort	comfortSafe = false is always a hard rejection.
Capacity	scheduledEnergyKwh <= maxShiftEnergyKwh.
Power	No interval schedule may imply power above maxPowerKw.
Target stop	Cumulative scheduled energy should not intentionally exceed target except explicit tolerance.
Baseline	Reference days marked eventRan=true are excluded from simple baseline.
Verification	verifiedFlexEnergyKwh cannot be negative.
Confidence gate	Settlement cannot be created when settlementAllowed=false.
Pool sum	Equity + contributor + reserve shares must sum to 1.
Equity Floor	Equity pool share must meet/exceed equityFloor.
Contributor cap	Total attributed contributor energy <= verified accommodated energy cap.
Credits	Sum of participant credits <= corresponding pool; cents only.
Idempotency	Applying the same settlement twice cannot create duplicate credits.

25. Recommended Persistence Model

For the 25-hour hackathon, start with seeded TypeScript/JSON. If persistence is added, map the same domain contract to PostgreSQL/Supabase rather than redesigning the model.

Table	Minimum columns
households	id, suburb, sunshine_cell_id, household_type, equity_eligible, wallet_id
participants	id, household_id, display_name, role, participation_status, consent_active
sunshine_cells	id, display_name, suburb, topology_ref, equity_score, confidence
resources	id, household_id, cell_id, type, max_power_kw, max_shift_energy_kwh, availability, compatibility, consent, comfort_safe, response_factor
solar_contributors	participant_id, household_id, cell_id, participation_status
flex_events	id, cell_id, start_time, end_time, forecast_export_kwh, comfortable_export_kwh, target_flex_kwh, confidence, status
dispatch_decisions	event_id, resource_id, decision, scheduled_kwh, weighted_score, reject_reason
verifications	event_id, baseline_kwh, observed_kwh, verified_flex_kwh, confidence, settlement_allowed
settlements	id, event_id, policy_id, verified_value_cents, equity_pool_cents, contributor_pool_cents, reserve_cents, status
credits	id, event_id, settlement_id, participant_id, type, amount_cents, status, idempotency_key

Hackathon recommendation

Do not add a database until the deterministic vertical slice works. The contract is designed so seeded JSON can be replaced by persistence later without changing engine/UI types.

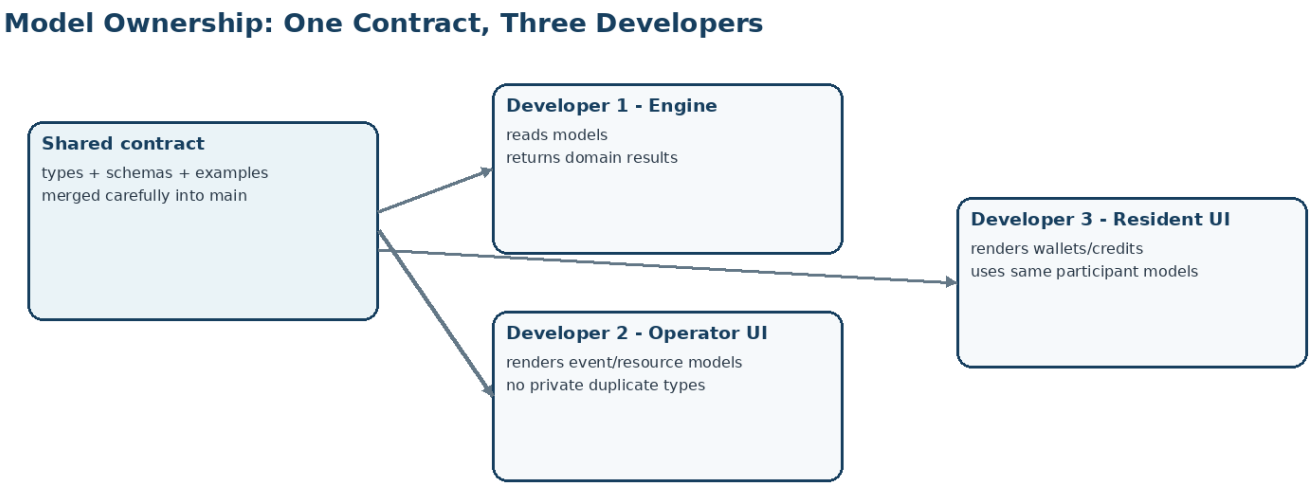
26. Folder Structure for Models

```
sunshine-wallet/
  src/
    domain/
      enums.ts
      participant.ts
      household.ts
      sunshine-cell.ts
      resource.ts
      contributor.ts
      event.ts
      dispatch.ts
      verification.ts
      settlement.ts
      credit.ts
      provenance.ts
      index.ts

    engine/
      select-event-window.ts
      check-resource-eligibility.ts
      optimise-resources.ts
      simulate-event.ts
      build-baseline.ts
      verify-event.ts
      attribute-contributors.ts
      settle-event.ts

    data/
      demo-scenario.ts
      reference-days.ts

    validation/
      model-guards.ts
      invariants.ts

    app/
      api/v1/...
      operator/...
      resident/...
```

Import rule

```
// Good
import type { FlexEvent, FlexibleResource, Credit } from '@/domain';

// Avoid
// Re-declaring local component-specific copies of these domain types.
```

27. Three-Person Ownership and Change Control



Owner	Consumes/owns	Rule
Developer 1 - Engine/API	Primary maintainer of domain contract + engine implementation.	May propose model changes; must flag breaking fields.
Developer 2 - Operator UI	Consumes FlexEvent, Resource, DispatchDecision, Verification, Settlement.	Never invents duplicate domain shapes.
Developer 3 - Resident/Contributor UI	Consumes Participant, WalletSummary, Credit, ContributorAttribution.	Never recalculates settlement locally.

Git rule for model changes

- Model contract files are high-conflict shared files; keep changes small.
- Open a short PR titled `contract: ...` for changes to shared models.
- Include a before/after JSON example when a field or enum changes.
- Do not merge a breaking model change while another developer has an unrebased feature branch consuming the old shape.
- After model PR merges: everyone pulls/rebases before continuing integration.

28. Contract Tests

Test	Setup	Expected
Wrong cell	HW-205 belongs to DAPTO-02; event is DAPTO-01.	Rejected, scheduled 0 kWh.
Unavailable EV	EV-04 available=false.	Rejected before scoring.
Partial final resource	19 kWh already selected, target 24, APT max 6.	PARTIALLY_SELECTED for 5 kWh.
Determinism	Same dispatch plan run twice.	Same 22.6 kWh simulated response.
Baseline contamination	Reference day has eventRan=true.	Excluded from baseline.
Low M&V confidence	confidence=0.63, threshold=0.70.	SETTLEMENT_BLOCKED, no credits.
Equity Floor invalid	equityPoolShare=0.50, floor=0.60.	Settlement rejected.
Contributor cap	Exports total 13, verified accommodation 10.	Attributable cap = 10.
Duplicate apply	Same Idempotency-Key sent twice.	One set of credits only.
No device equity user	Dinh has no compatible resource but equityEligible=true.	Can still receive Equity Dividend.

29. Full Worked Model Walkthrough

The complete DAPTO-01 scenario should be representable without any ad-hoc UI-only fields:

- DAPTO-01 forecast export = 70 kWh; comfortable export = 46 kWh; targetFlexEnergyKwh = 24 kWh; forecast confidence = 0.82.
- Eligible resources are evaluated. HW-101, HW-102, BAT-01 and APT-01 collectively schedule exactly 24 kWh. EV-04 is unavailable; HW-205 is in the wrong cell.
- Deterministic ResourceResponse records sum to 22.6 kWh actual simulated flexibility.
- BaselineEstimate uses comparable non-event days and returns 50.0 kWh. Observed event-period energy is 72.6 kWh, therefore Verification.verifiedFlexEnergyKwh = 22.6.
- M&V confidence 0.82 exceeds the 0.70 policy threshold, so settlementAllowed = true.
- Maria exported 6.5 of 13.0 qualifying contributor kWh (50%). If verified accommodated contributor energy is capped at 10 kWh, her attributedEnergyKwh = 5.0.
- At a simulated 80 cents/kWh, verified program value = 1,808 cents. Policy shares are 65% equity, 30% contributors, 5% reserve; the 60% Equity Floor is satisfied.
- Credits are ledger records: Dinh receives an Equity Dividend; Maria receives a Contributor Reward. The values shown in wallets come from Credit/WalletSummary, not hardcoded UI numbers.

Model contract success condition

A developer should be able to build the entire demo from these models without asking what “24 kWh,” “22.6 kWh,” Maria's contribution, the Equity Floor, or Dinh's bill credit mean.

30. Final Contract Checklist

- One shared `src/domain` package exists and all three developers import it.
- Every kWh/kW field name contains its unit.
- Rooftop solar contributor data is separate from dispatchable flexible resources.
- Every FlexEvent has a Sunshine Cell, time window, target, confidence and state.
- Every rejected resource stores a machine-readable reason.
- Scheduled response and actual/simulated response are separate models.
- Baseline reference days explicitly record whether a Wallet event ran.
- Verification owns the settlement confidence gate.
- Contributor attribution never claims electron tracing and is capped by verified outcome.
- Settlement policy is versioned and enforces the Equity Floor.
- Credits are integer cents and idempotent.
- Simulated, mocked, estimated and calculated data are distinguishable.
- UI screens display model results; they do not silently reproduce business logic.
- Contract tests cover the failure cases before final demo rehearsal.

Development philosophy

The model contract should make wrong states hard to represent. If a value has a different physical or business meaning, give it a different field/type instead of reusing a vague number.